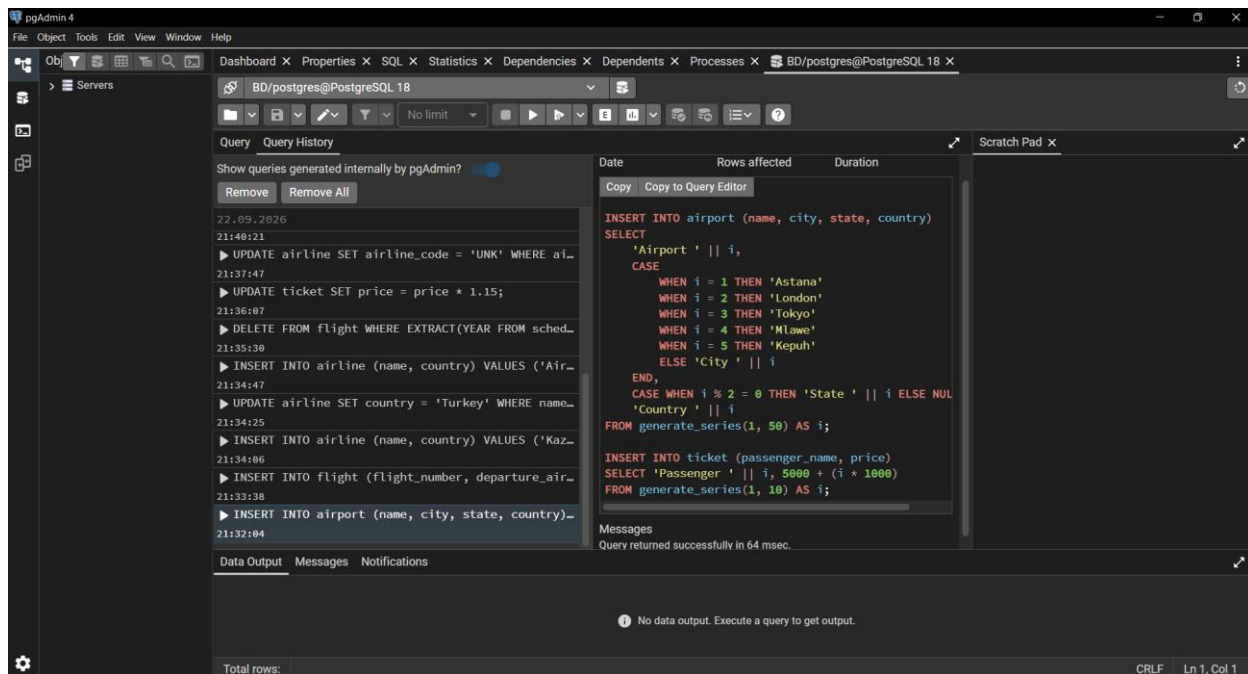
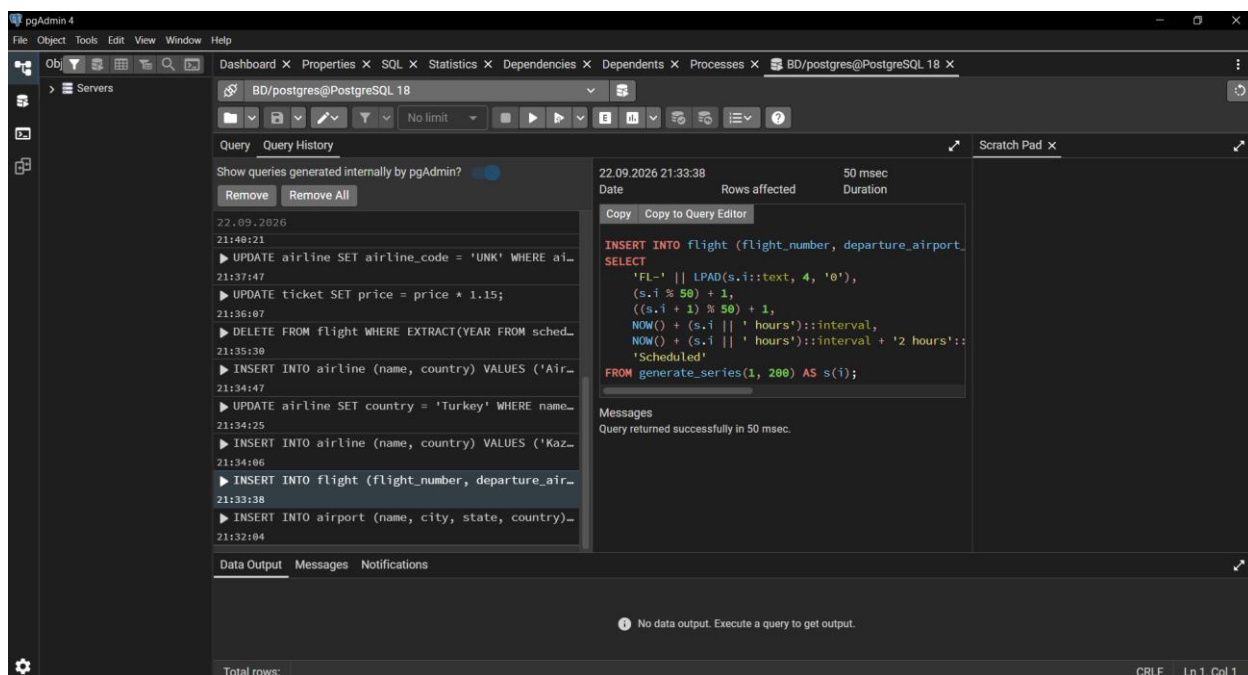


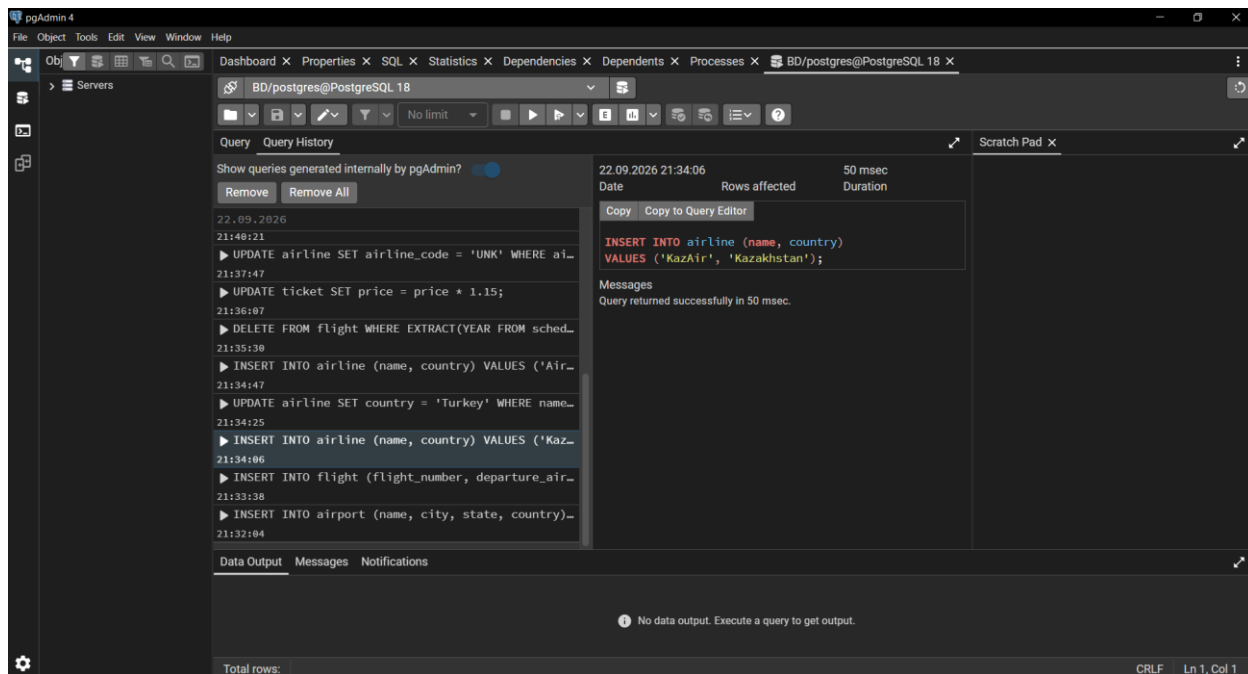


Airport Table Population The query inserts 50 records into the airport table using `generate_series(1, 50) AS i`. Names are created via `'Airport ' || i`. A CASE statement explicitly assigns required cities (Astana, London, Tokyo, Mlawe, Kepuh) to the first 5 rows and `'City ' || i` to the rest. The state column uses `i % 2 = 0` to assign states to even rows and NULL to odd rows, ensuring NULL values exist for future tasks.

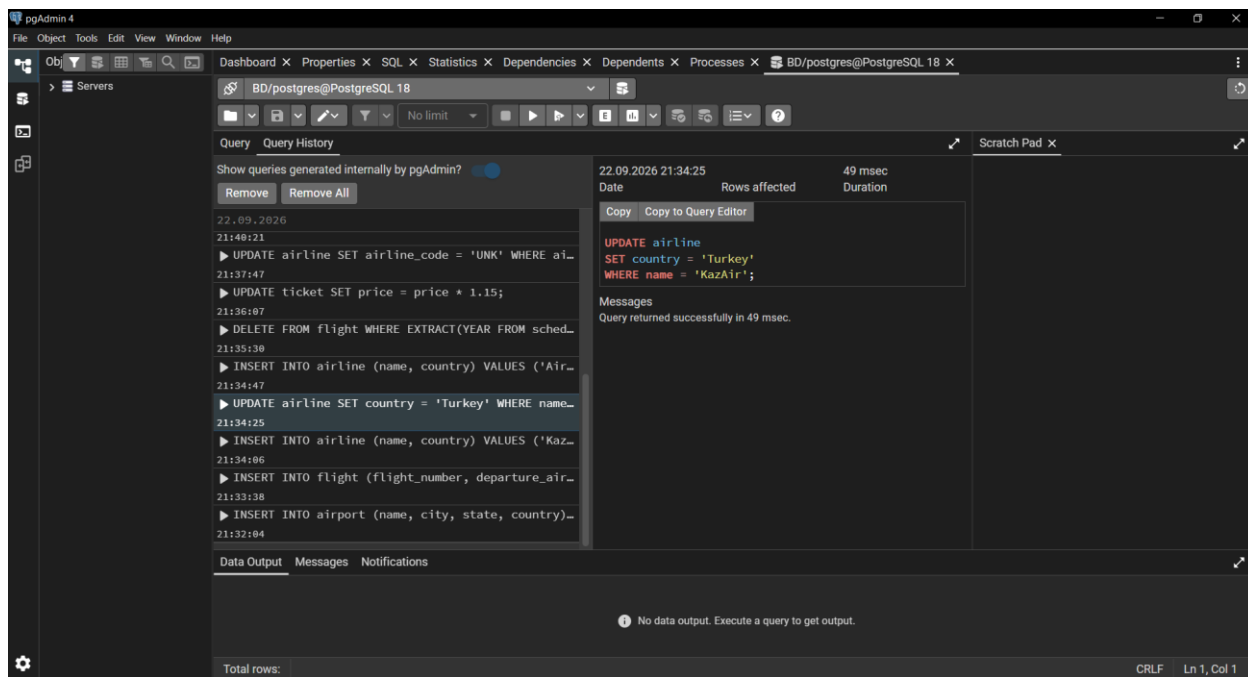
Ticket Table Population The query inserts 10 records into the ticket table using `generate_series(1, 10) AS i`. Passenger names are formatted as `'Passenger ' || i`, and prices are calculated using the formula `5000 + (i * 1000)`.



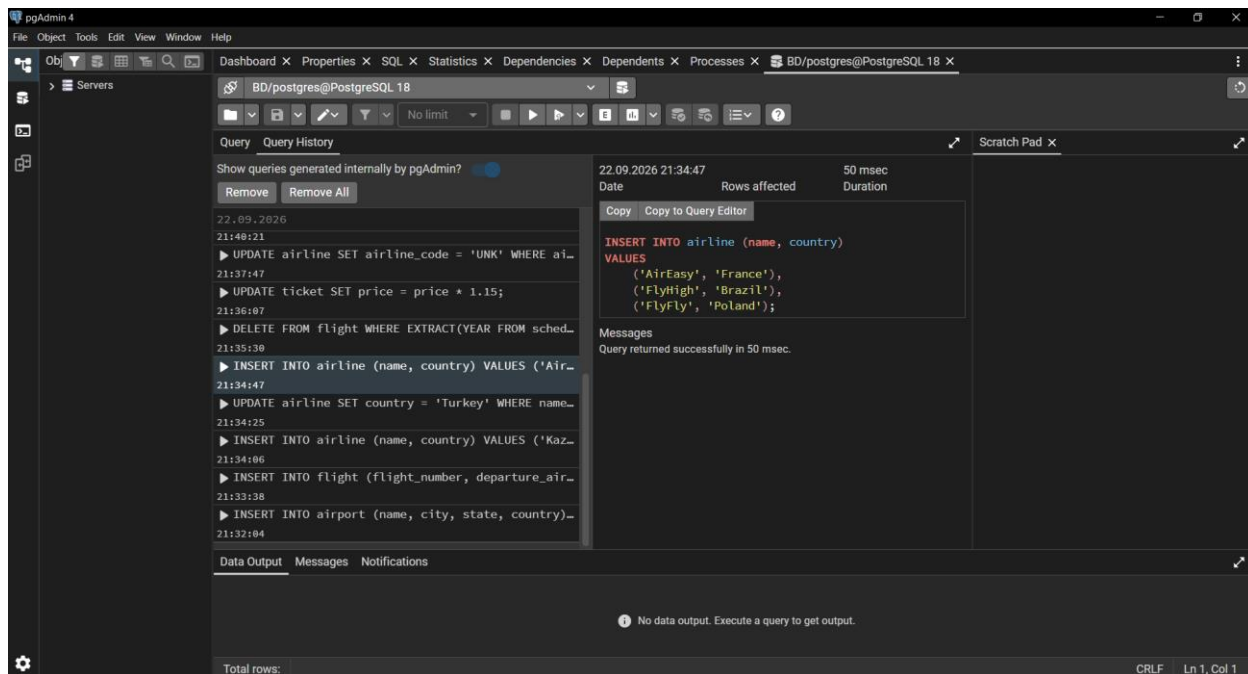
Flight Table Population The query generates and inserts 200 rows into the flight table using `generate_series(1, 200) AS s(i)`. Flight numbers are formatted as `'FL-' || LPAD(s.i::text, 4, '0')`. Departure and arrival airport IDs are linked to existing airports using modulo operations `(s.i % 50) + 1` and `((s.i + 1) % 50) + 1`. Departure times are dynamically calculated relative to `NOW()` by adding `(s.i || ' hours')::interval`, while arrival times add an extra `'2 hours'::interval`. The status is set to `'Scheduled'`.



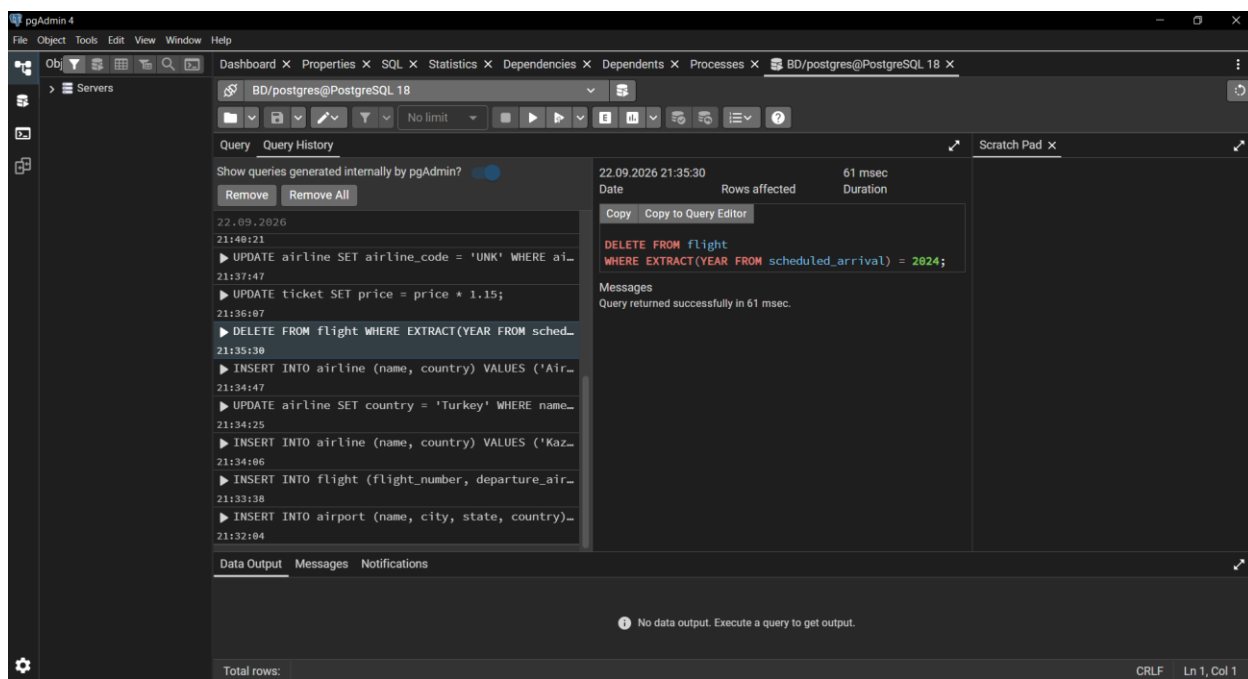
This is a basic INSERT INTO statement that adds a single row for 'KazAir' with country set to 'Kazakhstan'.



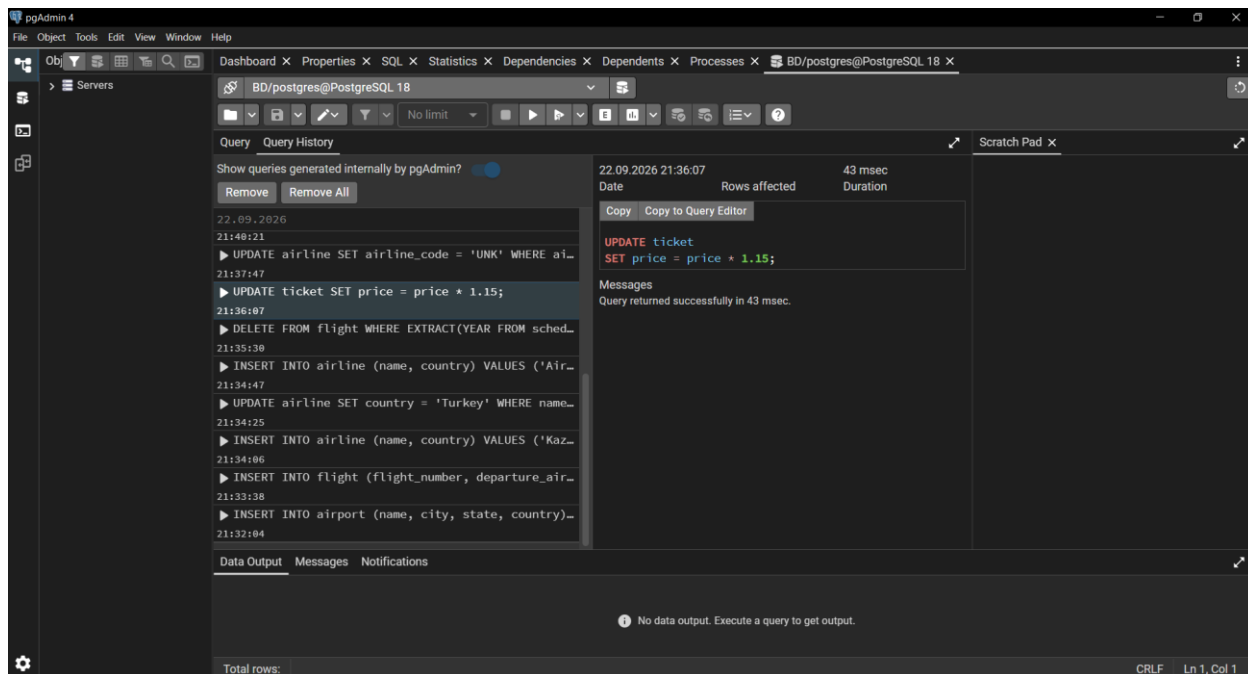
I used the UPDATE statement with a WHERE condition to modify the country of 'KazAir' specifically, changing it to 'Turkey'.



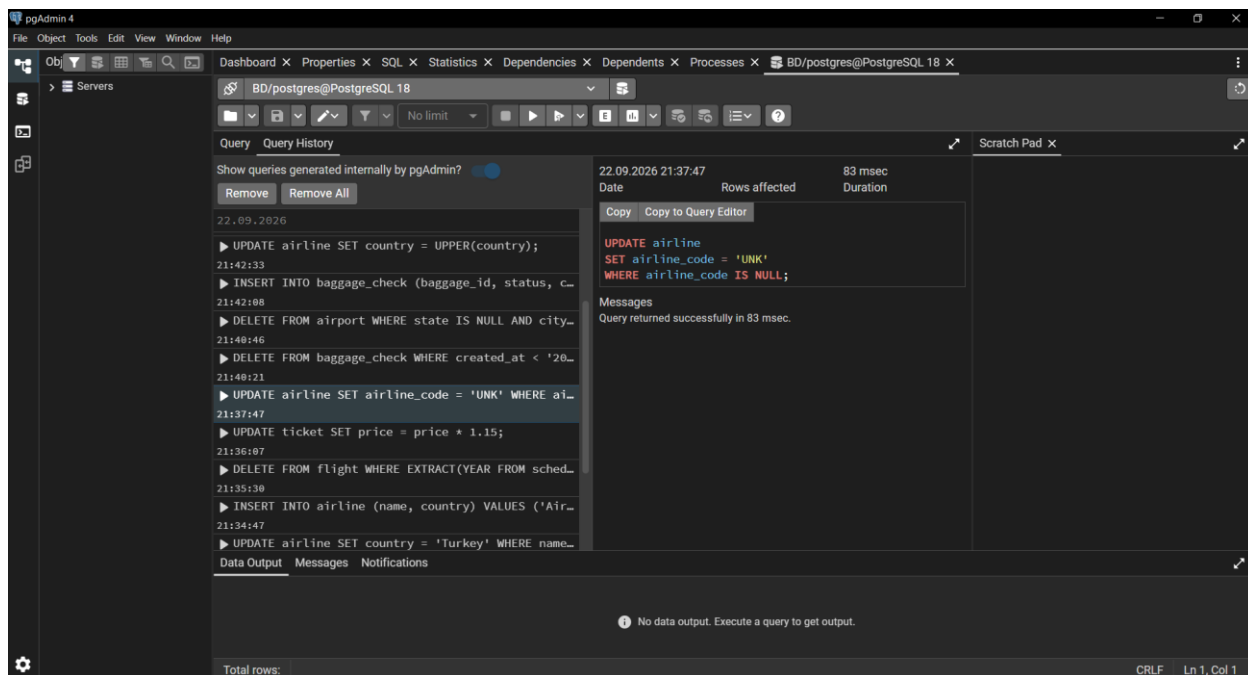
This is a bulk insert where multiple value tuples are passed in a single query to insert three airlines simultaneously.



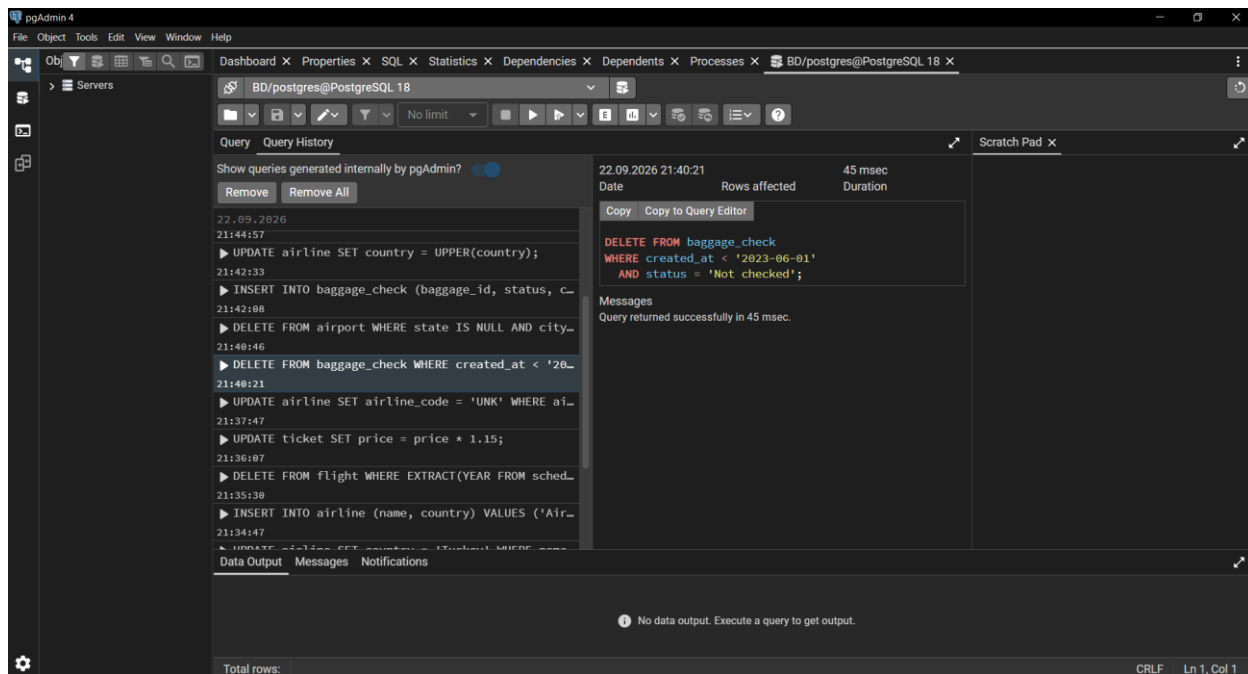
I used DELETE FROM combined with the EXTRACT(YEAR FROM ...) function to target and remove all flights scheduled to arrive in the year 2024.



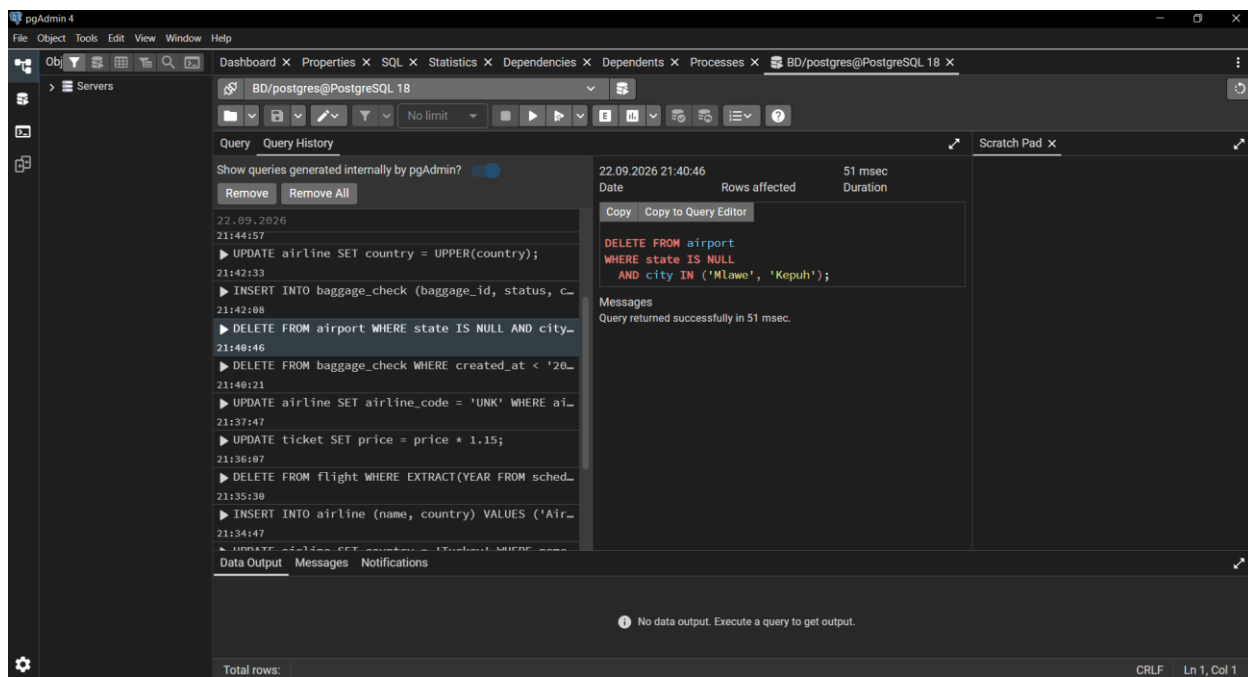
I updated all ticket prices by multiplying the current price column by 1.15, which effectively raises the price by 15%.



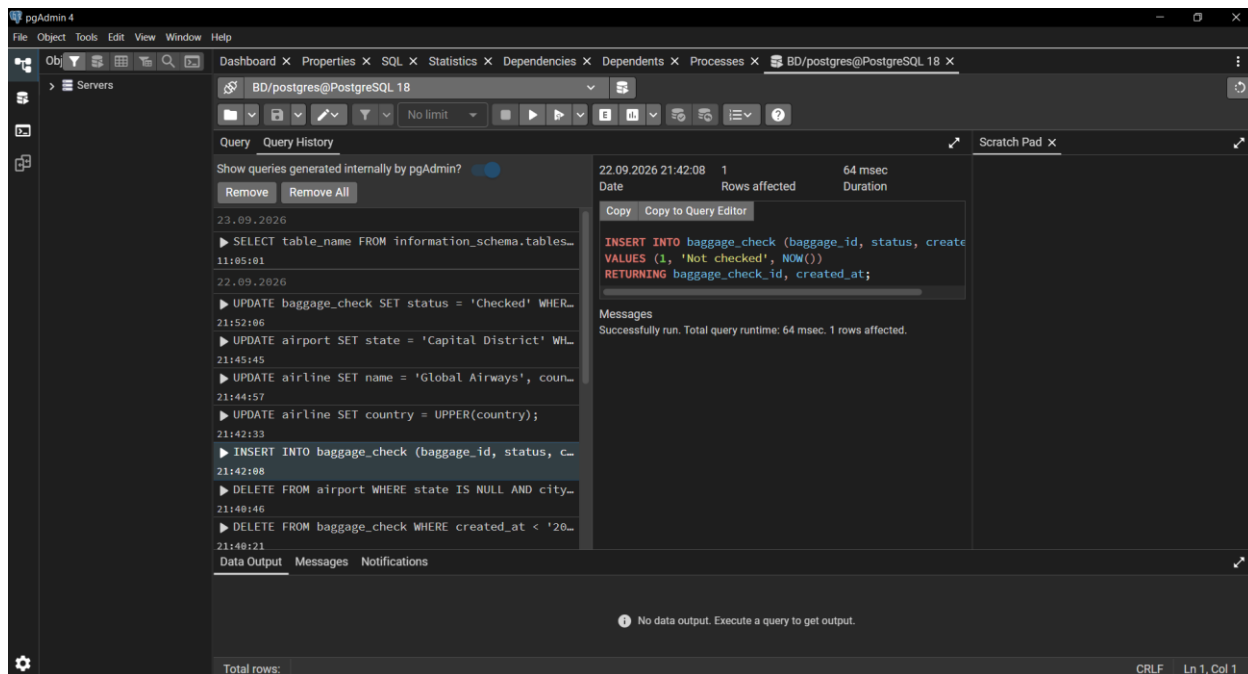
I used WHERE airline_code IS NULL to filter missing values and replaced them with the default code 'UNK'.



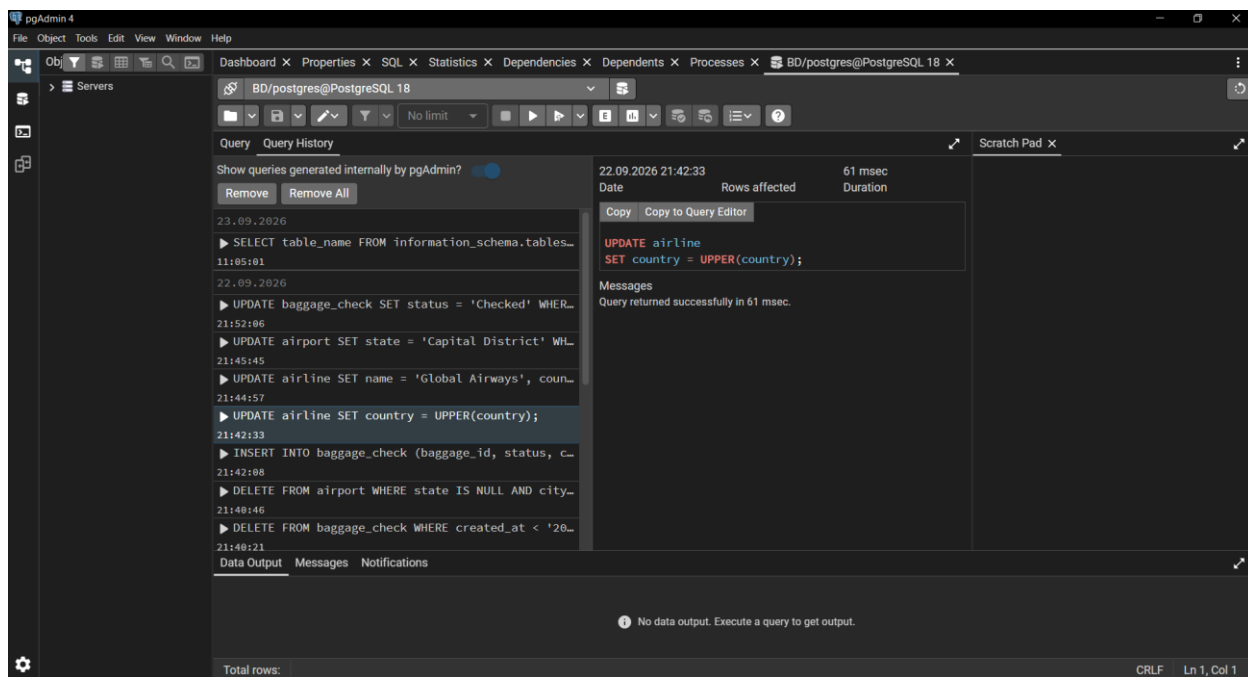
I combined two condition checks using AND to remove baggage inspections created before June 1, 2023 that still have a 'Not checked' status.



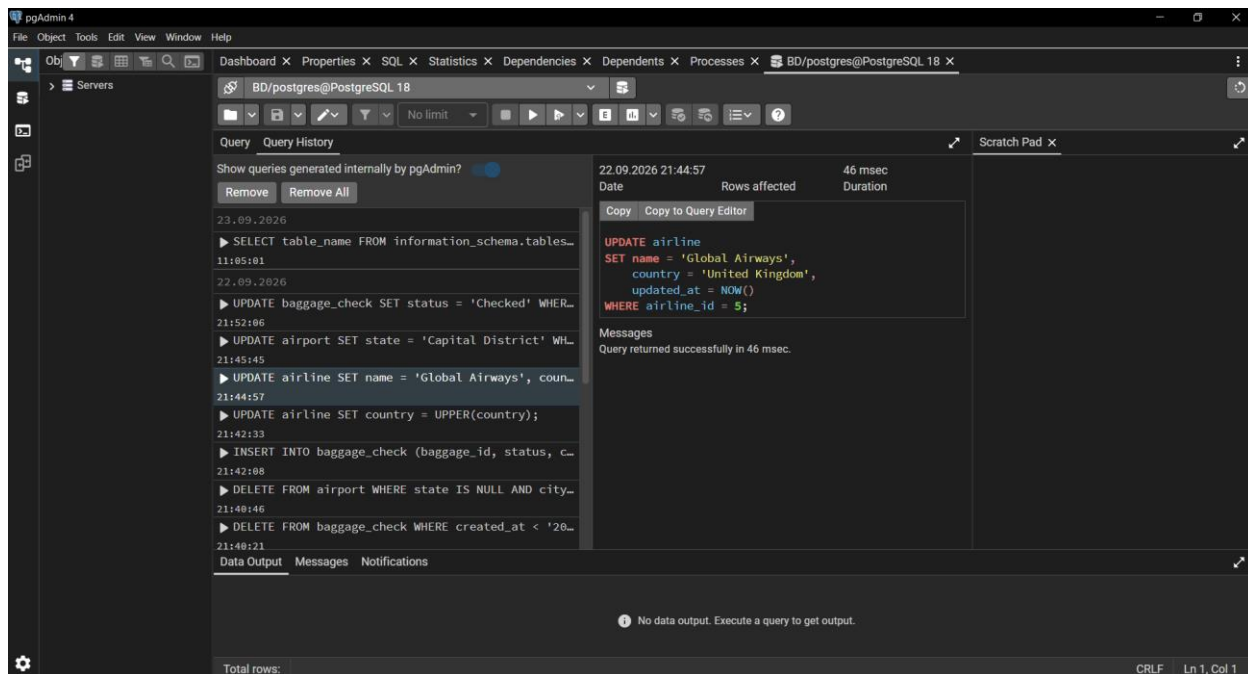
This query deletes airports where the state is NULL and the city matches either 'Mlawe' or 'Kepuh' using the IN operator.



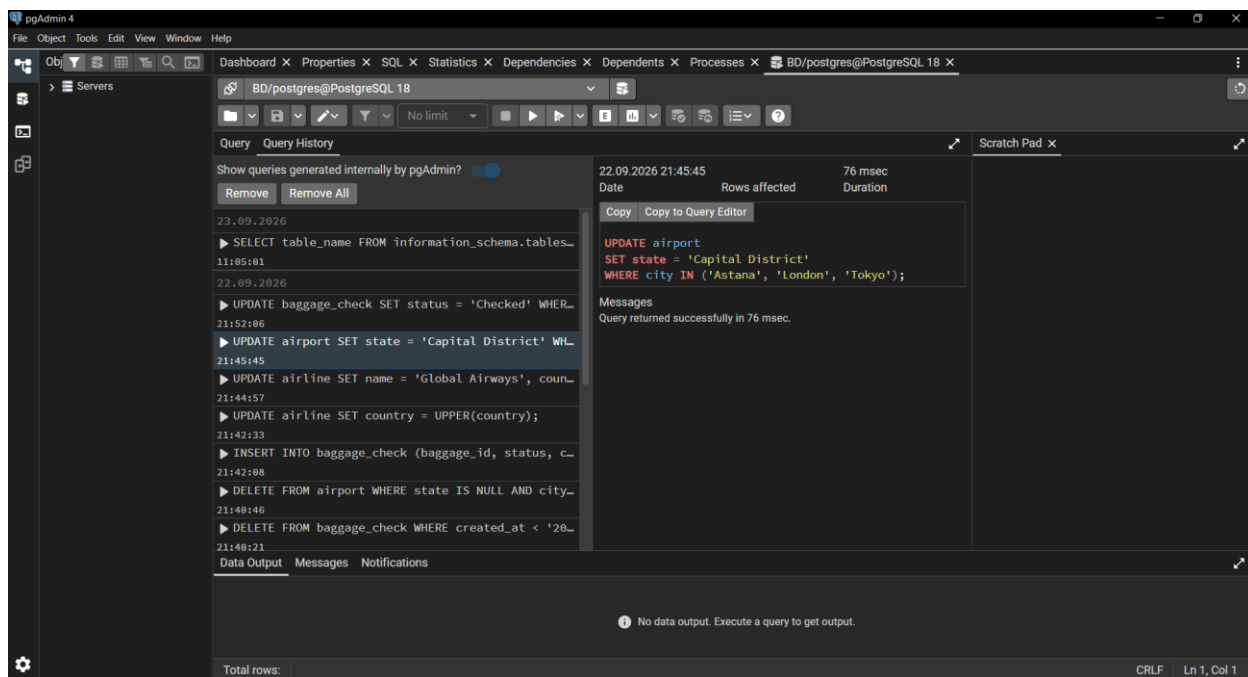
I inserted a new baggage check record and used the RETURNING clause to immediately receive the auto-generated ID and creation timestamp without running a separate SELECT query.



I applied the built-in UPPER() string function to normalize all country names in the airline table into uppercase letters.



I updated multiple columns at once for `airline_id = 5`, setting the name, country, and updating the timestamp with `NOW()`.



I filtered airports by city names using the `IN` clause and updated their state field to 'Capital District'.

The screenshot shows the pgAdmin 4 interface with the following components:

- Top Bar:** pgAdmin 4, File, Object, Tools, Edit, View, Window, Help.
- Tab Bar:** Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, Processes, BD/postgres@PostgreSQL 18.
- Left Panel:** Servers, showing a tree view of the database structure.
- Query History:** A list of queries with columns for Date, Rows affected, and Duration. The selected query is:

```
UPDATE baggage_check SET status = 'Checked' WHERE status = 'Not checked' AND created_at >= '2024-03-01' AND created_at < '2024-04-01';
```
- Messages:** Query returned successfully in 49 msec.
- Data Output:** No data output. Execute a query to get output.

To target March 2024, I set a date range filter between March 1st and April 1st, updating the status of all unchecked baggage to 'Checked'.